Ders 1 Genişletilmiş Notu: Nesneye Dayalı Tasarıma Giriş: Yazılım Kalitesi, Bağlaşım ve Uyum, Tasarım Kalıpları ve MVC

Ders 1 Genişletilmiş Notu: Nesneye Dayalı Tasarıma Giriş: Yazılım Kalitesi, Bağlaşım ve Uyum, Tasarım Kalıpları ve MVC

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Kapsamı ve Kaynakları
- 2. Ön Koşullar: Nesneye Yönelik Programlama ve UML
- 3. Yazılım Kalitesi
- 4. Yazılım Projelerinde Başarı
- 5. Bağlaşım (Coupling)
- 6. Bağlaşımı Ölçmek: İki Tasarımın Karşılaştırılması
- 7. Uyum (Cohesion) ve Veritabanı Sorumluluğu Örneği
- 8. Tasarım Kalıbı Nedir?
- 9. GoF Kalıplarının İlişkileri ve Sınıflandırması
- 10. Örnek Tasarım Kalıbı: Model-View-Controller
- Örnek Sorular

1. Dersin Kapsamı ve Kaynakları

Dersin ilk slaytında (ekrandan, 00:10:00) içerik iki kısım olarak verilir:

- Kısım 1: Gang of Four Tasarım Kalıpları
- Kısım 2: Code Smells and Refactoring (Birim sınamaları ile)

Dersin büyük kısmı tasarım kalıpları (design patterns) üzerine geçer. Bu kalıplar literatürde "Dörtlü Çete" (Gang of Four, GoF) adıyla anılan dört yazarın kitabında tanımlanmıştır: *Design Patterns: Elements of Reusable Object-Oriented Software*, Erich Gamma vd., Addison-Wesley, 1994. Literatürdeki klasik tasarım kalıpları bu kitaptadır ve dersin temel referansı da budur.

Hoca kitabın tarihinin kimseyi yanıltmaması gerektiğini söyledi. Tasarım kalıpları bir işin doğru, verimli ve yeterince iyi yapılmasına yönelik çözüm önerileridir ve bu öneriler nesneye dayalı modelleme üzerinedir. Nesneye dayalı programlamanın çekirdeği (parça-bütün ilişkileri, kalıtım vb.) 1960'larda ilk önerildiğinden beri pek değişmemiştir. Hoca ders notlarının önemli bir kısmını bu kitaptan oluşturmuş, ama her kalıbı pratik bir örnekle anlatmaya özen göstermiştir.

1.1 Ek kaynaklar

İkinci slaytta ek kaynaklar listelenir (ekrandan, 00:10:40):

- *Design Patterns Explained: A New Perspective on Object-Oriented Design*, Alan Shalloway ve James R. Trott, Addison-Wesley, 2004 (2nd Ed.)

- *Head First Design Patterns*, Elisabeth Freeman vd., O'Reilly, 2004
- *Design Patterns* (Wordware Applications Library), Christopher G. Lasater, Jones & Bartlett Publishers, 2006 (1st ed.)
- *Design Patterns Java Workbook*, S. John Metsker, Addison-Wesley, 2002
- *Applied Java Patterns*, S. Stelting ve O. Maassen, Prentice Hall, 2002

Hoca bu kitapların her birinden en az bir örneği ders notlarına eklemiştir. GoF kitabı kalıpları klasik tarzda anlatır; her kalıbın kullanımına örnek verir ama bu örnekler kısmidir, bütüncül bir örnek yoktur. Ayrıca hem o kitapta hem de ders notlarının çoğunda birden fazla kalıbın birlikte kullanıldığı örnekler bulunmaz. Ek kaynakların bazıları ise daha büyük bir proje çerçevesinde (örneğin bir öğrenci bilgi sistemi) bir gereksinim ortaya koyar, o gereksinime yönelik tasarımı yapar, "bu kalıba göre tasarlarsak şu avantajları elde ederiz" diye kalıbı uygular ve aynı anda birden fazla kalıbı da kullanır. Konuya iki yönden yaklaşmak iyi anlamak için yararlıdır. Hoca, öğrencilerin bulduğu iyi kaynakları da listeye eklemeye açık olduğunu söyledi.

1.2 Code smells ve refactoring

Tasarım kalıpları nasıl iyi tasarım yapılacağını söyler. Ama yanlışları bilmek ve onları nasıl düzelteceğimizin farkında olmak da yararlıdır. Burada code smells (kod kokuları) ve refactoring konuları devreye girer. Code smell kesinlikle bir bug değildir; uygun olmayan, esnek olmayan bir modellemedir. En küçük bir hata bulunduğunda ya da gereksinimlerde en küçük bir değişiklik olduğunda gereğinden çok fazla iş çıkarır. Bunların ne olduğunu bilip onlardan kaçınmak, karşılaşıldığında da düzeltmek refactoring'dir.

Bu kısmın temel referansı Martin Fowler'ın *Refactoring: Improving the Design of Existing Code* (Addison-Wesley, 1999) kitabıdır (ekrandan, 00:10:00). Hoca kitaptaki code smell'lerin yaklaşık yarısını, kendi kanaatine göre en çok karşılaşılanları seçerek ders notlarına almıştır. Fowler'ın web sitesinde de bu konuda kaynak bulunur.

Dersin çekirdeği bu yüzden şöyle özetlenebilir: nesneye dayalı tasarım ve modellemeyi (a) nasıl iyi yapacağımız, (b) kötü yapıldıysa nasıl düzelteceğimiz.

2. Ön Koşullar: Nesneye Yönelik Programlama ve UML

Bu derste ilerleyebilmek için nesneye dayalı programlamanın temellerini bilmek ve bunları ödevlerde, projelerde, çalışmalarda kullanmış olmak gerekir. Slayttaki gereksinimler (ekrandan, 00:10:40) şöyledir:

- Herhangi bir güncel NYP diline hakimiyet. Derste kod örnekleri Java dili üzerinden verilecektir.
- UML sınıf şemalarına hakimiyet. Bu konuda çok eksik öğrenciler olabilir; şu kaynaklardan biri incelenmelidir: *UML Distilled*, 3rd ed. (2003), Martin Fowler, Addison-Wesley; *UML 2.0 in a Nutshell*, Dan Pilone ve Neil Pitman, O'Reilly; vb.

Hoca örnekleri Java ile verecek, ama işin özü modelleme olduğu için Java'ya özgü hiçbir özellik kullanmayacak. Bu yüzden Java kodu, baştaki `package` kelimesi `namespace` ile değiştirilerek doğrudan C#'a aktarılabilir. C++ bilenler de küçük bir notasyon farkıyla zorlanmaz.

2.1 UML sınıf şemaları

Hocanın en sık karşılaştığı sorun UML sınıf şemalarına hakimiyettir. Bunun kaynağı bilgisayar mühendisliği müfredatlarındaki bir eksikliktir. UML (Unified Modeling Language), bir yazılım oluştururken yapılan analiz ve tasarım etkinliklerini teknik olarak belgelemeye yönelik, sektörde standart bir dildir ve kendi içinde birçok şema türü içerir. Bu derste tasarımlar sınıf şemaları üzerinden anlatılır. Nesneye dayalı bir tasarımı göstermenin en yaygın, neredeyse tek yolu UML sınıf şemalarıdır. Bu yüzden UML'e hakim olmayan öğrenciler mutlaka UML çalışmalıdır.

Hoca özellikle *UML Distilled*'ı önerdi ("distilled": özümsenmiş, damıtılmış). İnce bir kitaptır ve yalnızca UML anlatmakla kalmaz; her şema türünü ve bileşenini anlatırken o kavram hakkında da bilgi verir. Örneğin kalıtımın UML'de içi boş üçgen okla gösterildiğini söylerken kalıtımın anlamını, çok biçimlilikle kullanımını, ne zaman doğru ne zaman yanlış kullanıldığını da kısaca anlatır. Şart değildir; O'Reilly'nin "in a Nutshell" serisi de birçok konuda iyidir.

Hoca, lisans düzeyinde verdiği Nesneye Dayalı Programlama dersinin notlarını da önerdi. Türkçe ve İngilizce literatürde NYP dilini öğreten kitaplar ile UML'i anlatan kitaplar ayrıdır; üniversite müfredatlarında da durum benzerdir. Hoca ise bu iki konunun iç içe geçmiş olduğunu ve bütüncül anlatılması gerektiğini düşünür. Bu yüzden NYP dersinde koda yakın şema türleri olan sınıf şemaları ve sequence (sıralama) şemaları dille birlikte anlatılır. Diğer UML şema türlerini merak edenler hocanın yazılım mühendisliği ders notlarına bakabilir; orada hangi şemanın yazılım geliştirme sürecinin hangi aşamasında, hangi amaçla kullanıldığı anlatılır.

3. Yazılım Kalitesi

Nesneye dayalı tasarım ve modellemeyi nasıl daha iyi yaparız sorusu üzerine çalışıyoruz. Tasarım kalıpları bunu doğrudan söyler; code smells ise yanlışı gösterip doğruya nasıl gidileceğini anlatır. Amaç daha kaliteli bir yazılım üretmektir ve bu ancak iyi bir tasarım üzerine yapılabilir. "Ne kadar iyi?" sorusuna objektif ölçütlerle yanıt verebilmek gerekir.

Slayttaki "Yazılım Kalitesi" başlığı (ekrandan, 00:40:00) şöyledir:

- 'İyi' tasarım ile 'kaliteli' yazılıma ulaşmak amaçlanır.
  - Problem alanı hangi düzeyde parçalara ayrılabiliyorsa, o düzeyde parçalara ayrılabilen bir tasarım da 'iyi' olarak nitelendirilebilir.
- Ölçütler: dış kalite özellikleri (kullanıcıya yönelik), iç kalite özellikleri (programcıya yönelik).
- Dış kalite ölçütleri: doğruluk, etkinlik, kolaylık, …
- İç kalite özellikleri: yeniden kullanılabilirlik, ilgi alanlarının ayrılması, esneklik, taşınabilirlik, okunabilirlik ve anlaşılabilirlik, sınanabilirlik, bakım kolaylığı, …

Hoca iyi tasarımın bu tanımını çok sevdiğini, ama tanımın somut metriklerle ölçülebilir olmadığını, bu yüzden daha objektif bir kritere ihtiyaç olduğunu söyledi.

Kalite farklı kişilerin farklı yaklaştığı bir konudur. Dış kalite özellikleri son kullanıcı için önemlidir: programın doğru girdiye karşı doğru sonuç vermesi (doğruluk), sistem kaynaklarını verimli kullanması (etkinlik), son kullanıcının rahatça kullanabilmesi (kolaylık). Bu derste ise iç kalite özelliklerine yönelik konular işlenir. Tasarım kalıpları da refactoring de iç kalite özelliklerini iyileştirmeye yarayan konulardır.

3.1 İç kalite özellikleri

- **Yeniden kullanılabilirlik:** bir projede yapılan tasarımın ya da kodun bir parçasının başka projelerde de kullanılabilmesi.
- **İlgi alanlarının ayrılması (separation of concerns):** bir sınıfa ya da kod parçasına sorumluluk atanırken, o parçanın gerçek hayatta karşılık düştüğü varlığın sorumluluklarına uygun sorumluluklar atanması. Yazılım projesinin gereksinimleri ve bakış açıları birbirine karmaşık, iç içe geçmiş biçimde değil, mümkün olduğunca temiz hatlarla ayrılmış biçimde kodlanmalıdır.
- **Esneklik:** kodun bir parçası başka bir projeye taşınacağı zaman bunun mümkün olduğunca az değişiklikle yapılabilmesi.
- **Taşınabilirlik (portability):** kodun bir platformdan başka bir platforma mümkün olduğunca az değişiklikle taşınabilmesi.
- **Okunabilirlik ve anlaşılabilirlik:** kodun kolay okunması ve bir kod bloğunun ne iş yaptığının kolay anlaşılması.
- **Sınanabilirlik:** yazılımın kolay test edilebilmesi; belirli bir işi test etmek için başka şeyleri de test etmek zorunda kalmamak.
- **Bakım kolaylığı.**

  Hocanın vurgusu: Okunabilirlik ve anlaşılabilirlik çok önemlidir, çünkü doğrudan bakım kolaylığıyla ilgilidir. Bakım, yazılımdaki hataları düzeltme ya da yeni gereksinimleri ekleme etkinlikleridir. Yazılım sahaya sunulduktan sonra bakım aşamasına geçilir ve bu aşamada da değişiklik yapmak gerekir. Bir yazılım projesinde harcanan çaba, emek, zaman ve maliyetin, hangi taraftan bakılırsa bakılsın, rahatça yarısı bakım aşamasında harcanır. Bu yüzden bakım kolaylığını artıran her şeye sıkı sıkı sarılmak gerekir. Kod rahat okunamıyorsa anlamaya sıfırın gerisinden başlanır.

Hoca bu konuya refactoring kısmında daha ayrıntılı döneceğini söyledi.

4. Yazılım Projelerinde Başarı

Neden iyi tasarıma bu kadar önem veriyoruz? Çünkü yazılım projeleri önemli oranda başarısız olur. Bunun tabii başka birçok etkeni vardır: değişken sektör şartları, krizler, zaman kestiriminde yanlış kararlar, fazla iyimser planlamalar. Bunlar her zaman denetlenemez, ama iyi bir tasarım yapıp yapmadığımızı denetleyebilir ve bunun için elimizden gelen çabayı gösterebiliriz.

Yazılımın temel dinamiği şudur: oluşan bir hatayı düzeltmenin bedeli, yazılımın yaşam döngüsünde ilerledikçe üstel olarak artar. O yüzden hataları mümkün olduğunca erken bulmak gerekir.

4.1 Başarı tanımı ve oranlar

Hoca ders notlarında olmayan, görece güncel bir çalışmayı gösterdi (ekrandan, 00:42:40). Slayt, "Form Tabanlı Uygulamalar İçin Çaba Kestirimi" başlıklı bir UYMS 2015 sunumundandır ve kaynak olarak "Dobb's Journal 2013 IT Project Success Survey" yazılıdır.

Bir yazılım projesine başarılı diyebilmek için üç koşul vardır. Proje:

1. önceden belirlenmiş zamanda,
2. önceden belirlenmiş bütçeyle,
3. önceden belirlenmiş gereksinimleri karşılayacak biçimde

tamamlanmış olmalıdır. Bu üçünden birinde bile olumsuz sapma varsa proje başarısız sayılır. Bu sıkı tanımla, 2013'teki uluslararası çalışmada kullanılan sürece göre farklılık göstermekle birlikte, geleneksel şelale (waterfall) süreciyle yürütülen projelerin yaklaşık %50'si başarılı olur; yani iki projeden biri başarısızdır. Çevik (agile) süreçlerle bu oran %65 civarına çıkar. Çevik süreçlerin üzerine kurulmuş Lean Development yaklaşımında ise %70'ler civarındadır.

Grafikte (ekrandan, 00:42:40 ve 00:43:45) Lean, Iterative, Agile, Ad-Hoc ve Geleneksel süreçler için "Başarılı", "Zorlanılmış" ve "Başarısız" oranları yatay çubuklarla gösterilir. "Zorlanılmış" slaytta "bütçe, süre, işlev yönlerinden en azından birinde başarısızlık" diye tanımlanır. Geleneksel süreç için fare ile gösterilen değerler: Zorlanılmış %32, Başarısız %18.

Hoca daha esnek bir başarı tanımı da yaptı: üç faktörden birindeki olumsuz sapma göz ardı edilir ve diğer ikisi sapmadan ulaşılırsa proje "kısmi başarılı" sayılır. Bu durumda başarılı ve kısmi başarılı projelerin oranı geleneksel süreç için %80-82'ye, çevik süreçler için %90'a çıkar. Geleneksel süreç için bu, grafikteki değerlerle tutarlıdır: $100 - 18 = 82$.

Sonuç: işi baştan sıkı tutmak ve doğru adımlarla ilerlemek gerekir. Özellikle şelale süreciyle çalışılıyorsa en başta düzgün ve iyi bir tasarım yapmak zorunludur; aksi hâlde başarısızlığa davetiye çıkarılır.

5. Bağlaşım (Coupling)

İyi tasarım ilkelerini somutlaştırmak için hoca bağlaşım ve uyum kavramlarını ele aldı. İyi bir tasarımda **düşük bağlaşım** (low coupling) ve **yüksek uyum** (high cohesion) olmalıdır. Bu iki ilke birbiriyle ilişkilidir: genelde düşük bağlaşım yüksek uyumu, yüksek uyum da düşük bağlaşımı beraberinde getirir.

Bağlaşım, bir parçanın diğer parçalarla ilişkide bulunma oranıdır. Hoca "bağlaşım, bağımlılık" gibi birbirine benzeyen kelimelerle tanım yapmaktan kaçındığını, bu yüzden "bağımlılık" yerine "herhangi bir ilişki" demeyi tercih ettiğini söyledi. Nesneye dayalı yazılımda bağlaşım sınıf, alt sistem ya da paket düzeyinde düşünülebilir; derste temel olarak sınıf düzeyinde bağlaşım ele alınır.

İki sınıf arasında şu durumlarda bağlaşım vardır:

- bir sınıf diğerinin hizmetlerinden (metotlarından) yararlanıyorsa,
- diğerinin iç yapısından ya da çalışma prensiplerinden haberdarsa,
- aralarında kalıtım ilişkisi varsa.

Esas mesele şudur: ilişkide bulunulan diğer sınıfların sayısı arttıkça bağlaşım oranı artar.

Düşük bağlaşımın kazancı şudur: bir sınıfta yapılan değişikliğin geri kalan sınıfları etkileme olasılığı azalır. İster istemez hatalar bulunacak ya da gereksinimler değişecek, ilgili bir ya da birkaç sınıfta değişiklik yapılacaktır. Bu değişiklik daha önce doğru çalışan bazı sınıfların ya da metotların yanlış çalışmasına yol açabilir. Bağlaşım düşükse ilişkide bulunulan sınıf sayısı az olacağı için bu olumsuz etkilenme olasılığı da düşer.

Tabii ki sıfır bağlaşım olamaz. Ne kadar bağlaşımın kaçınılmaz olduğu önceden bilinemez, problem alanına göre değişir. Önemli olan gereksiz yere fazladan ilişki kurmamaktır.

6. Bağlaşımı Ölçmek: İki Tasarımın Karşılaştırılması

6.1 Violet UML Editor

Hoca örneği Violet UML Editor programında çizdi (ekrandan, 00:50:00; dosya `VioletUML-0.21.1`, çalıştırılabilir jar). Program sınıf şemalarını ve use case, state, activity, sequence gibi diğer UML şemalarını sürükle-bırak ile çizmeye, saklamaya, sonra açıp değiştirmeye ve PNG olarak dışa aktarmaya yarar. Kod üretme ya da koddan şema çizme yeteneği yoktur. Küçük bir jar dosyası olduğu için Java Runtime Environment kurulu olmalıdır. Programın yeni sürümü eski sürümle uyumsuz yapıldığı için hoca, eski sürümle çizdiği çok sayıda şema nedeniyle eski sürümü kullanmaya devam eder; yetenek açısından büyük bir fark yoktur ve öğrenciler yeni sürümü indirebilir.

6.2 Küçük bir UML hatırlatması

A sınıfından B sınıfına düz bir ok (association) varsa, A sınıfı içinde B sınıfı türünden en az bir üye alan vardır. Üye alanlar private tanımlanır; UML'de private `-` ile gösterilir. Şemada A sınıfının içine bu üye alan `- b : B` olarak da yazılabilir (ekrandan, 00:52:40). Okun ucundaki `1` çokluk (multiplicity) bilgisidir.

6.3 İki tasarım

Aynı gereksinimleri karşılayan iki tasarım çizildi (ekrandan, 00:52:40):

```
Tasarım (I)                    Tasarım (II)

  A ---1--> B                     A ---1--> B
            |                     |
            1                     +---1--> C
            v
            C
```

```
A'da "- b : B" üye alanı var.
```

Soru sınıfa soruldu: düşük bağlaşım daha iyiyse hangi tasarım seçilmeli? Öğrenciler Roma rakamıyla oy verdi. Ekrandaki notlara göre (ekrandan, 01:00:00) Tasarım I'e 2, Tasarım II'ye 8 oy geldi.

6.4 Doğrudan ilişkilerle ölçüm

Önce sistemin toplam bağlaşımı sayıldı:

- Tasarım I: A, B'ye bağımlı; B, C'ye bağımlı. Bağlaşım(I) = 2.
- Tasarım II: A, B'ye bağımlı; A, C'ye bağımlı. Bağlaşım(II) = 2.

Toplamlar aynı olduğu için sınıf bazında ayrıntıya inildi. Ekrandaki notlar (ekrandan, 01:00:00):

```
(I): 2 oy                      (II): 8 oy
Bağlaşım(I) = 2                Bağlaşım(II) = 2
Bağ(A) = 1                     Bağ(A) = 2
Bağ(B) = 1                     Bağ(B) = 0
```

```
Bağ(C) = 0                      Bağ(C) = 0
Worst case: 1 (+)               Worst case: 2 (-)
```

C hiçbir sınıfa bağlı olmadığı için Bağ(C) = 0'dır. En kötü durum (worst case) açısından bakılırsa Tasarım I'de en çok bağlaşımlı sınıfın bağlaşımı 1, Tasarım II'de 2'dir. Bu ölçüye göre hoca Tasarım I'i tercih etmeye meylettiğini söyledi.

Soru: Bir öğrenci Tasarım II'yi şöyle savundu: C sınıfında bir sorun çıktığında Tasarım I'de hem A hem B bozulur, Tasarım II'de ise B etkilenmez. **Cevap:** Hoca bunu mantıklı buldu.

Soru: Bir başka öğrenci, B'yi Tasarım I'den çıkarmak istediğimizde A ve C'nin etkileneceğini, Tasarım II'de ise yalnız A-B bağının etkileneceğini söyledi. **Cevap:** Hoca benzer bir bakış açısı olduğunu, ama okun yönünün önemli olduğunu düzeltti: A'dan B'ye ok olduğu için A, B'ye bağımlıdır, B ise A'ya bağımlı değildir. A'da bir değişiklik olduğunda B etkilenmez.

6.5 Dolaylı ilişkilerle ölçüm

Hoca şimdiye kadar yalnız doğrudan ilişkileri saydığını hatırlattı. Ölçütü nasıl tanımladığımıza bağlı olarak dolaylı ilişkiler de sayılabilir. Tasarım I'de A, B'ye; B de C'ye bağımlı olduğu için aslında A, C'ye de dolaylı olarak bağımlıdır. Bu dolaylı bağlantı yarım bağlantı (0,5) sayılırsa (ekrandan, 01:00:00):

```
(I)                             (II)
Bağlaşım(I) = 2,5               Bağlaşım(II) = 2
Bağ(A) = 1,5                    Bağ(A) = 2
Bağ(B) = 1                      Bağ(B) = 0
Bağ(C) = 0                      Bağ(C) = 0
Worst case: 1 (+)               Worst case: 2 (-)
```

Hesap: Bağ$(A) = 1 + 0{,}5 = 1{,}5$ ve Bağlaşım$(I) = 1{,}5 + 1 + 0 = 2{,}5$. Tasarım II'de dolaylı ilişki yoktur, toplam 2 kalır. Bu ölçüyle, en kötü duruma hiç bakmadan, Tasarım II tercih edilebilir.

> **Hocanın vurgusu:** Somut sayılarla çalışsak bile ölçütümüzü nasıl tanımladığımız sonucu değiştirir.

Hoca bir noktayı daha ekledi: "A ne, B ne, C ne?" diye sorulabilirdi. A, B, C sınıflarından oluşturulan nesneler, çözmeye çalıştığımız gerçek dünya problemindeki varlıklara karşılık gelir. Gerçek dünyadaki ilişki Tasarım I'deki gibiyse, diğerine göre ne kadar iyi ya da kötü olursa olsun Tasarım I seçilmelidir, çünkü gerçek dünyadaki durum budur. Hoca sonraki örnekte Tasarım I'i kullandı.

7. Uyum (Cohesion) ve Veritabanı Sorumluluğu Örneği

Uyum, bir parçanın sorumluluklarının birbirleriyle uyumlu olma oranıdır. Yani bir parçanın sahip olduğu sorumlulukların birbirine benzemesi, "ayrı dünyaların sorumlulukları" olmamasıdır.

Uyum yükseldikçe:

- sınıf daha kolay anlaşılır,
- yeniden kullanılabilirlik artar,
- bakım kolaylığı artar,
- sınıfın değişikliklerden etkilenme olasılığı düşer.

Son maddenin nedeni şudur: yüksek uyum, sınıfın sorumluluklarının birbirine yakın ve odaklanmış olması demektir. Gereksinimlerde bir değişiklik olduğunda bu değişiklik yalnız onunla ilgili sınıfı değiştirir; yüksek uyumlu sınıfımızda o farklı gereksinime yönelik bir şey bulunmadığı için bu sınıf değişmez. Genelde düşük bağlaşım getiren bir tasarım yüksek uyumu (iyi), yüksek bağlaşım getiren bir tasarım düşük uyumu (kötü) beraberinde getirir.

7.1 Varlıklar veritabanına kendilerini mi kaydetsin?

Tasarım I'deki A, B, C varlıklarının ister istemez veritabanına kaydedilmesi gerekir. Violet'te veritabanı sembolü olmadığı için hoca veritabanını paket sembolüyle (VT) gösterdi. İki seçenek çizildi (ekrandan, 01:05:40):

```
Seçenek 1: Her varlık kendi kaydından sorumlu
  A ---1--> B ---1--> C
  A - - -> VT
  B - - -> VT
  C - - -> VT

Seçenek 2: Ayrı bir kalıcılık sınıfı D
  A ---1--> B ---1--> C
  D - - -> A
  D - - -> B
  D - - -> C
  D - - -> VT

(--->  düz ok: üye alan / association)
(- - > kesikli ok: kullanma / dependency)
```

Seçenek 2'de D sınıfı nesneleri veritabanına yazmaktan ve veritabanından okumaktan sorumludur. Veritabanına erişir, kayıtları okur, ilgili sınıfların yapıcılarını (constructor) çağırarak nesneleri oluşturur. Bunun için D ile A, B, C arasında sahiplik olmasa bile en azından **kullanma ilişkisi** olmalıdır. Kullanma ilişkisi kesikli okla gösterilir ve arada üye alan olmadan, bir sınıfın bir metot içinde geçici değişken olarak tanımlanması ya da bir metoda parametre olarak aktarılması anlamına gelir. Tasarıma göre sahiplik de olabilir.

Sınıfa yine soruldu ve gelen bütün oylar Seçenek 2 yönündeydi.

Hoca bunun üzerine bağlaşımı hesapladı. Tasarım I'in A, B, C arasındaki bağlaşımı 2 (ya da dolaylı ilişki sayılırsa 2,5) idi. Seçenek 2'de D'den A, B, C'ye üç kullanma ilişkisi eklendi; bunlar yarım sayılsa bile:

$$\text{Bağ}(II) = 2 + 3 \times 0{,}5 = 3{,}5$$

Ekranda da notlar `Bağ(I)=2-2,5` ve `Bağ(II)=3,5` olarak yazılıdır (ekrandan, 01:10:00). Hoca "şeytanın avukatlığı" yaparak bağlaşım arttığı hâlde neden Seçenek 2'nin seçildiğini sordu.

Soru (sohbet penceresinden): Bir öğrenci "single responsibility" yazdı. **Cevap:** Hoca doğru olduğunu, burada ilgi alanlarının ayrılması (separation of concerns) ve uyum üzerine konuşulduğunu söyledi.

Soru: Bir öğrenci sözlü olarak, sınıfların her birinin veritabanına erişmesi yerine bunun tek bir sınıf üzerinden halledilmesinin hem karmaşıklığı azalttığını hem de tek sorumluluk ilkesine uyduğunu söyledi. **Cevap:** Hoca doğruladı.

7.2 Veritabanı değişirse ne olur?

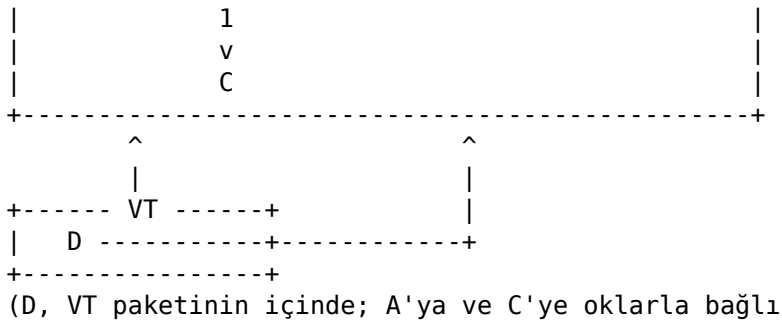
Hoca gerekçeyi şöyle somutlaştırdı. Veritabanı diyelim ki MySQL olsun. Ama veri grafiksel ya da uzamsal bir veriyse (bir graf, bir harita ve haritadaki yerler, aradaki yollar), MySQL'den Oracle'a ya da Microsoft SQL Server'a geçmek bir yana, bu tür verileri iyi işleyen başka bir sisteme, örneğin PostgreSQL'e ya da ilişkisel olmayan bir veritabanı sistemine geçmek gerekebilir (ekrandan, 01:10:00: şemada `VT:MySQL` ve `PostgreSQL` paketleri). Bu durumda yazma ve okuma işlemlerinin hepsinin içeriği değişir.

- Seçenek 1'de bu kodu A, B ve C sınıflarında ayrı ayrı değiştirmek gerekir.
- Seçenek 2'de yalnız D sınıfını değiştirmek yeterlidir. A, B, C kullanılan veritabanı altyapısından bağımsız kalır.

7.3 Katmanlama

Seçenek 2 bir tür katmanlama da sağlar. Hoca şemayı iki paketle yeniden düzenledi (ekrandan, 01:11:40):

```
+------------------ İş Mantığı ------------------+
|   A ---1--> B                                  |
|             |                                  |
```



(D, VT paketinin içinde; A'ya ve C'ye oklarla bağlı)

A, B, C sınıfları iş mantığı katmanı paketinde, D ise veritabanı (kalıcılık) katmanı paketinde yer alır. Böylece iş mantığı katmanı ile uygulama altyapısı katmanı birbirinden ayrılmış olur. Hoca katmanlı yapının normalde PowerPoint'te üst üste dikdörtgenler ve aralarındaki oklarla daha kolay çizildiğini söyledi.

> **Hocanın vurgusu:** Bu istisnai bir örnektir. Bağlaşım kötüleşti, ama uyumu daha yüksek olan tasarımı seçtik. Genelde ise düşük bağlaşım yüksek uyumu, yüksek bağlaşım düşük uyumu beraberinde getirir.

8. Tasarım Kalıbı Nedir?

Aradan sonra hoca tasarım kalıplarına giriş yaptı. Slayttaki tanım (ekrandan, 01:20:00):

- Yazılım tasarımında karşılaşılan belirli sorunların yalın ve güzel çözümlerinin tarifidir.
  - Çözüm somut bir gerçekleme olmak zorunda değildir, soyut bir tarif de olabilir.
- Amaç: Tekerleği yeniden keşfetmeden 'iyi' tasarıma ulaşmak.
  - Sorunların esnek biçimde ve yeniden kullanılabilirliği arttıracak yönde çözülmesi.

Kalıp deyince genelde akla bir gerçekleme önerisi gelir, ama kalıp daha soyut bir tarif de olabilir.

8.1 Mimarlıktan gelen kavram

Bilgisayar mühendisliği, özellikle yazılım mühendisliği görece yeni bir meslektir. Bu yüzden daha eski disiplinlerden kavramlar ödünç alınıp uyarlanır. Tasarım kalıbı da ilk olarak mimarlıkta öne sürülmüş bir kavramdır. Hocanın verdiği örnek: bir bina tasarlanırken, içinde birden fazla insanın on dakikadan fazla zaman geçireceği her bölümde, odada ya da alanda mutlaka doğal aydınlatma, yani pencere olmalıdır. Hoca rakamları tam doğru söylemiyor olabileceğini, örneğin genel olduğunu belirtti. Bu mantıklı bir öneridir: uzun süre çalışan insanların verimli ve moralli olabilmesi için güneş ışığına ve ara sıra dışarıya bakıp nefes alabilmeye ihtiyacı vardır.

Yazılım mühendisliğinde ise tasarım kalıplarının amacı, tasarım problemlerine esnek, yeniden kullanılabilirliği artıran, uyumu yüksek ve bağlaşımı düşük tutan çözüm önerileri getirmektir. Böylece tekerleği yeniden keşfetmeden önceki deneyimden yararlanılır.

8.2 Bir tasarım kalıbının dört bileşeni

Bir şeye "tasarım kalıbı" diyebilmek için dört parçası olmalıdır (ekrandan, 01:20:00):

1. **İsim:** Kalıbı en güzel şekilde anlatacak bir ya da birkaç kelimelik isim. Güzel isim vermek kolay değildir; genelde çalışma belli bir noktaya geldikten sonra geri dönülüp uygun isim bulunur. İsim yalnız estetik bir kaygı değildir: tasarım kalıpları yazılım ekibi arasında ortak bir sözlük oluşturur, bu yüzden kalıbın niteliğini çağrıştıran bir isim çok önemlidir.
2. **Sorun:** Kalıbın çözdüğü sorunun tarifi.
3. **Çözüm:** Sorunu çözmek için önerilen tasarımın bileşenleri; bu bileşenlerin ilişkileri, sorumlulukları, birlikte nasıl çalıştıkları.
4. **Tartışma/Sonuçlar:** Çözümün sistemin geneline esneklik, genişletilebilirlik, taşınabilirlik gibi yönlerden yaptığı etkiler; çözümün güçlü ve zayıf yönleri; yer-zaman çelişkileri (trade-off); başka kalıplar ile işbirliği olanakları vb.

Yer-zaman çelişkisine hocanın örneği: bir kalıbı kodlamakta izlenebilecek birkaç yol varsa, biri bellek optimizasyonuna, diğeri işlemci optimizasyonuna yönelik olabilir. Bu tür ödünleşimler tartışma bölümünde anlatılır. GoF kitabında her kalıbın bu dört bileşeni yeterli zenginlikte ve nitelikte anlatılır.

8.3 Tasarım kalıplarını çalışmanın kazançları

- Tekerleği yeniden icat etmemeyi sağlar. Kalıplar birçok deneyimli kişinin emeğiyle bu hâle geldiği için kaliteli çözümler olduklarından emin olunabilir. Ama kalıp doğru uygulanmazsa ya da "illa bu kalıbı uygulayacağız" diye düşünülürse, yazılımın gereksinimlerine ters düşen ya da esnekliğini azaltan bir iş de yapılabilir.
- Deneyimli kişilerin deneyimini yazılı biçimde aktarır. Deneyim genelde ancak çalışarak, deneme ve öğrenme süreciyle edinilir. Kalıpları önerenler yıllarca o problemle ve türevleriyle karşılaşmış, genel ve iyileştirilmiş bir çözüme ulaşmış; sonraki yıllarda bu çözümler doğrulanmıştır.
- Yazılım ekibi arasında ortak bir sözlük oluşturur. Hocanın örneği: yeni bir projenin analiz aşamasında biri "farklı vergi kuralları olduğu için her bölgenin vergi kurallarını birer Strategy örneği yapabiliriz" der. Diğeri "bazı küçük ek kural parçacıkları aynı ülke içinde bile değişebiliyor; o zaman bazı strateji örnekleri Builder kalıbıyla kurulmalı" diye ekler. İki cümleyle 10-12 sınıflık bir tasarım anlatılmış olur. Program alanını bilen tasarımcılar bunu daha da az kelimeyle aktarabilir.

Hoca, bazı kalıpları gördükten sonra öğrencilerin "bu bir kalıp değil, şu iki temel ilkenin bir araya getirilmesi" ya da "daha basit bir ilke" diyebileceğini, bunda haklı olabileceklerini, ama bunların literatüre tasarım kalıbı olarak geçtiğini söyledi.

9. GoF Kalıplarının İlişkileri ve Sınıflandırması

9.1 Kalıplar arası ilişki şeması

GoF kitabında bütün tasarım kalıplarının ve aralarındaki birlikte çalışma olasılıklarının açıklamalı oklarla gösterildiği bir şema vardır (ekrandan, 01:21:50). Şemada yer alan kalıplar: Builder, Iterator, Memento, Adapter, Proxy, Bridge, Command, Composite, Decorator, Flyweight, Visitor, Interpreter, Chain of Responsibility, Strategy, State, Mediator, Observer, Template Method, Prototype, Factory Method, Abstract Factory, Facade, Singleton. Okların bazı açıklamaları:

```
Composite        --creating composites-->        Builder
Composite        --enumerating children-->       Iterator
Iterator         --saving state of iteration-->  Memento
Composite        --adding responsibilities
                   to objects-->                 Decorator
Decorator        --changing skin versus guts--   Strategy
Strategy         --sharing strategies-->         Flyweight
State            --sharing states-->             Flyweight
Interpreter      --sharing terminal symbols-->   Flyweight
Composite        --adding operations-->          Visitor
Interpreter      --defining grammar-->           Composite
Observer         --complex dependency
                   management-->                 Mediator
Template Method  --defining algorithm's steps--> Strategy
Template Method  --often uses-->                 Factory Method
Abstract Factory --configure factory
                   dynamically-->                Prototype
Abstract Factory --implement using-->            Factory Method
Abstract Factory --single instance-->            Singleton
Facade           --single instance-->            Singleton
```

Bu şemayı ezberlemeye kesinlikle gerek yoktur; gerektiğinde kitapta vardır. Hocanın şemadan verdiği örnek: Strategy kalıbı kullanılırken stratejiler başka bileşenlerle paylaşılacaksa

Flyweight olarak kodlanabilir. Kitapta kalıbın tarifinde stratejilerin her birinin bir Singleton olmaya aday olduğu da yazılır, ama bu ilişki şemada çizilmemiştir.

Derste Interpreter hariç GoF kalıplarının hepsi işlenecektir.

9.2 Amaçlarına göre üç küme

Slayttaki sınıflandırma (ekrandan, 01:27:20):

- **Creational (Yaratımsal):** Nesnelerin oluşturulması, temsili ve ilişkilendirilmelerindeki bağımlılığı azaltmaya yönelik kalıplardır.
- **Structural (Yapısal):** Sınıflar ve nesnelerin bir araya getirilerek daha büyük yapılar elde edilmesine yönelik kalıplardır.
- **Behavioral (Davranışsal):** Sorumlulukların nesnelere dağıtılması ve algoritma seçimine yönelik kalıplardır.

GoF kitabı kalıpları bu sırayla anlatır: önce yaratımsal, sonra yapısal, sonra davranışsal kalıplar. GoF yazarları kalıpları başka açılardan da gruplamıştır; örneğin varlıklar arasındaki bağlaşımı azaltmaya yönelik kalıplar, nesnelerin temsilini sınıf gösteriminden ayırmaya yarayan kalıplar, sorumlulukların daha iyi atanmasına yarayan kalıplar. Ama anlatımda yaratımsal-yapısal-davranışsal sırasını tercih etmişlerdir.

Hoca ders notlarında başta farklı bir sıra kullanmıştı. Ancak bir kalıbı anlatırken başka bir kalıba gönderme yapmak gerektiğinden (çapraz referanslar) sıralama sorunları çıkıyordu; ayrıca dersi klasik kitaptan ya da benzerlerinden takip etmek isteyen öğrenciler için karışıklık oluyordu ve baktığı kaynakların yaklaşık %90'ı kitabın sırasını izliyordu. Bu yüzden ders notları artık kaynak kitaptaki sıralamaya sadık kalır.

9.3 Sınıfsal ve nesnesel kalıplar

GoF'a göre bazı kalıpların odağı sınıflar, bazılarının odağı nesnelerdir (ekrandan, 01:30:00):

- **Sınıfsal kalıplar:** Sınıflar ve bunların alt sınıfları arasındaki ilişkilere yönelik kalıplardır. Sınıf yapısı ve kalıtım ilişkisinin doğası gereği derleme zamanında belirlenen ve daha durağan ilişkilerdir.
- **Nesnesel kalıplar:** Çeşitli türden nesneler arasındaki ilişkilere yönelik kalıplardır. Kodun çalışması süresince rahatlıkla değiştirilebilen ve daha dinamik ilişkilerdir.
- Adapter kalıbı GoF tarafından hem sınıfsal hem de nesnesel bir kalıp olarak değerlendirilmiştir. Diğer kalıplar bu iki gruptan yalnızca birine dahil edilmiştir.

10. Örnek Tasarım Kalıbı: Model-View-Controller

Giriş bölümünün örnek kalıbı Model-View-Controller'dır (MVC). Hoca MVC deyince bazılarının aklına Microsoft'un gelebileceğini, ama Microsoft'un MVC'sinin bu tasarım kalıbının üzerine inşa edilmiş bir framework olduğunu ve adını da bu yüzden aldığını söyledi.

> **Ders sonrası not:** Hoca MVC'yi "Dörtlü Çete'nin kalıbı" diye tanıttı. MVC büyük olasılıkla GoF'un 23 kalıbından biri değildir; GoF kitabının giriş bölümünde Smalltalk MVC örneği olarak, Observer, Composite ve Strategy kalıplarıyla ilişkilendirilerek anlatılır.

10.1 Sorun ve çözüm

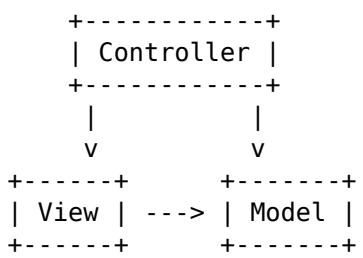
Slayttaki tarif (ekrandan, 01:31:45 ve 01:35:10):

- Sorun:
  - Aynı veriyi farklı biçimlerde gösterebilmek istiyoruz.
  - Kullanıcıya, verinin nasıl ve ne kadarının gösterileceğini seçebilme gibi olanaklar sağlamak istiyoruz.
- Çözüm:
  - Veriyi, verinin gösterimini ve gösterimin yönetimini ayrı sınıflar şeklinde ele almak.
  - Model sınıfı: ham veriyi simgeler.
  - View sınıfı: verinin çeşitli gösterim biçimleri.

- Controller sınıfı: kullanıcı komutlarını alıp işler.

Kalıp adını bu üç sınıftan alır. Controller'ın işlediği komutlar "veriyi şu biçimde göster" ya da "verinin şu kısmını göster" gibi olabilir; bazı kullanıcılara veriyi değiştirme hakkı da verilebilir.

Slayttaki temel sınıf şeması:



Yani Controller → View, Controller → Model ve View → Model ilişkileri vardır.

10.2 Seçim örneği

Hoca bu örneği genelde tahtaya yazdığını söyledi ve Excel'de gösterdi. Bir seçimde A partisi %40, B partisi %30, C partisi %10, diğerleri %20 oy almış olsun. Bu veri bir veritabanında, hatta belki bir metin dosyasında durur. Ama veriyi çubuk grafikle ya da pasta grafikle, hatta "diğer"in açılımını da vererek (D, E, F, G partilerinin oyları tek tek) göstermek isteriz. "İlk üç partiyi göster, kalanları birleştir" ya da "ilk n partiyi göster" gibi, verinin ne kadarının ve hangi biçimde gösterileceğini seçebilmek isteriz. Excel bunu tıkladıkça değiştirme esnekliğini verir; modellemeyi de buna göre yapmak isteriz.

Daha sonra hoca C'nin oyunu %12 yaptı; diğerleri buna bağlı olarak %18 oldu (ekrandan, 01:40:00: tablo A %40, B %30, C %12, Diğer %18 ve aynı değerleri gösteren sütun grafiği). Tablodaki değer değişince grafik otomatik olarak güncellendi.

10.3 Model değişince View nasıl haberdar olur?

Kalıbın temel yapısı yukarıdaki gibidir, ama GoF tartışma kısmında bazı durumları ele alır. Diyelim ki veride bir değişiklik oldu: bir sandık daha açıldı ve oy oranları değişti. View bunu nasıl anlayacak?

Değişikliği yapan taraf View'u ya da Controller'ı haberdar etmiyorsa ya da edemiyorsa (gerekli ağ haberleşmesi, altyapı ya da tümleştirme çalışması çok maliyetliyse veya başka nedenlerle yapılamıyorsa) iki yol vardır:

1. View belli aralıklarla modeli yoklayıp değişip değişmediğini kontrol eder. Bu aralık çok kısa seçilirse veri değişmediği hâlde View sürekli iş yapar, sisteme ek yük getirir ve kullanıcıya sunulan arayüzün yanıt süresi kötüleşebilir.
2. Model ben değiştim diyebilmelidir: Model'den View'a da bir ilişki kurulur, yani View ile Model arasındaki ilişki iki yönlü olur.

İkinci yol bağlaşımı yükseltir. Önceden View → Model, Controller → View ve Controller → Model olmak üzere toplam bağlaşım 3 idi; Model → View eklenince 4 olur. Bunu View'un daha az iş yapması için yaptık. Normalde model, kendisinin nasıl ve hangi parçasının gösterildiğinden bağımsızdır; bunlar veriyi ilgilendirmemelidir.

> **Hocanın vurgusu:** Tasarım kalıbı bir dogma, emir ya da yasak değildir; kendi ihtiyaçlarınıza göre değiştirebileceğiniz bir çözüm önerisidir. View'u hafif (lightweight) tutmanız gerekiyorsa bağlaşımı o kadar dert etmeyin ve Model'den View'a oku çizin. Öyle bir durum yoksa o bağlaşımı boşuna koymayın.

Aynı veriye birden fazla View bakıyor olabilir. O zaman ileride görülecek Observer kalıbı devreye girebilir.

> **Ders sonrası not:** Hoca bu kalıbı "observable tasarım kalıbı" diye andı; GoF kitabındaki adı Observer'dır.

10.4 Kullanıcı veriyi View üzerinden değiştirirse

Excel örneğinde hoca değişikliği klavyeyle, hücrenin içine tıklayarak girdi. Bu kutular Controller'ın değil View'un, yani gösterimin parçasıdır. Dolayısıyla değişiklik komutu View'a girilmiş olur. View'da yeni veri güncellemesi için `updateSomething()` gibi bir metot varsa, değişiklik modele aktarıldıktan sonra Controller'ın da bundan haberdar edilmesi gerekebilir. Çünkü başka View'lar da olabilir; Controller diğer View'ların `refresh()` metodunu çağırarak onların da güncel veriyi göstermesini sağlamalıdır. Bu durumda View'dan Controller'a da bir ilişki koymak zorunlu olur ve bağlaşım daha da yükselir. Bu artış hesaplama, iletişim ya da haberleşme maliyeti açısından kazanç sağlıyorsa zorunlu bir yatırım olarak görülebilir.

Controller birden fazla View ile ilişkili olabildiği için ilişkinin View ucuna çokluk yazılır (ekrandan, 01:41:15). UML'de * sıfır ya da daha fazla demektir. Alt limit belirtilecekse "alt limit, iki nokta, üst limit" biçiminde yazılır: `1..*` bir ya da daha fazla demektir.

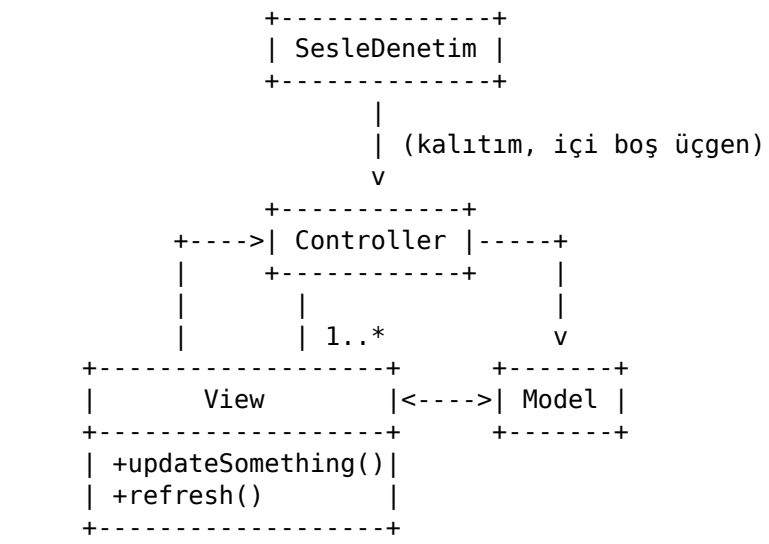
10.5 Ressam benzetmesi

Hoca sınıf adlarının boşuna seçilmediğini, kalıbın bir ressamın bir insanın resmini çizmesine benzetilebileceğini söyledi. Ressamın resmini çizdiği kişiye "model" denir; veri de odur. Controller ressamdır: modeli nasıl ve ne kadar çizeceğine o karar verir. Tüm boy ya da yalnız omuzdan yukarısı (portre) çizebilir; gerçekçi, karikatür (örneğin burnu ya da göbeği abartılı) ya da sürrealist biçimde çizebilir. Ortaya çıkan resim View'dur. Gösterimin nasıl olacağı ressamın kontrolündedir.

10.6 Kalıbın genişletilmesi

View'lar iç içe (nested) olabilir ya da çok iş parçacıklı (multi-threaded) çalışabilir; bu durumlarda farklı çözümler düşünülebilir. Kalıp "mutlaka şöyle yapacaksın" diye bir şey dayatmaz. Controller da tek olmak zorunda değildir: komutlar klavyeyle, fareyle ya da ikisinin birleşimiyle verilebilir. Sonradan sesle denetim özelliği eklemek istenirse yeni bir Controller sınıfı ortaya çıkar. Bu durumda kalıtım kullanılabilir: mevcut Controller sınıfı üst sınıf (parent) olarak tutulur ve ondan `SesleDenetim` sınıfı türetilir. Kalıtım okunun yönü alt sınıftan üst sınıfa doğrudur.

Ekrandaki son şema (ekrandan, 01:45:25):




```
Controller <-> View: iki yönlü (View tarafında 1..*)
View <-> Model:      iki yönlü
Controller -> Model: tek yönlü
```

Örnek Sorular

Soru 1 (ekrandan, 00:52:40; derste sözlü, 00:52:38): Aynı gereksinimleri karşılayan iki tasarım veriliyor. Tasarım I: A → B → C (A'da B türünden, B'de C türünden üye alan). Tasarım II: A → B ve A → C. Düşük bağlaşım daha iyi olduğuna göre hangi tasarım seçilmelidir?

> **Çözüm (derste verildi):** Yalnız doğrudan ilişkiler sayılırsa iki tasarımın toplam bağlaşımı da 2'dir. Sınıf bazında Tasarım I'de Bağ(A) = 1, Bağ(B) = 1, Bağ(C) = 0, en kötü durum 1; Tasarım II'de Bağ(A) = 2, Bağ(B) = 0, Bağ(C) = 0, en kötü durum 2. En kötü duruma göre Tasarım I daha iyidir. Dolaylı A → C ilişkisi 0,5 sayılırsa Tasarım I'in bağlaşımı 2,5 olur, Tasarım II'ninki 2 kalır; bu ölçüyle Tasarım II daha iyidir. Sonuç ölçütün tanımına bağlıdır; ayrıca gerçek dünyadaki ilişki Tasarım I gibiyse Tasarım I seçilmelidir.

Soru 2 (ekrandan, 01:05:40; derste sözlü, 01:04:15): A, B, C varlıklarının veritabanına kaydedilmesi ve yüklenmesi gerekiyor. Bu sorumluluk varlıkların kendisinde mi olmalı, yoksa nesneleri veritabanına yazmaktan ve okumaktan sorumlu ayrı bir D sınıfı mı oluşturulmalı? D sınıfının eklenmesi bağlaşımı nasıl etkiler?

> **Çözüm (derste verildi):** Ayrı D sınıfı seçilmelidir. D, A, B, C ile en azından kullanma (dependency) ilişkisi kurar; bu üç ilişki 0,5 sayılırsa bağlaşım $2 + 3 \times 0{,}5 = 3{,}5$ olur, yani artar. Ama uyum yükselir (tek sorumluluk, ilgi alanlarının ayrılması): veritabanı MySQL'den PostgreSQL'e ya da ilişkisel olmayan bir sisteme geçerse yalnız D değişir, A, B, C altyapıdan bağımsız kalır ve iş mantığı katmanı ile kalıcılık katmanı ayrılır.

Soru 3 (derste sözlü, 01:36:21): MVC kalıbında modeldeki veri değiştiğinde (örneğin bir sandık daha açıldığında) View bu değişikliği nasıl anlayabilir?

> **Çözüm (derste verildi):** Ya View belli aralıklarla modeli yoklar (aralık kısa olursa sisteme gereksiz yük biner, yanıt süresi kötüleşir) ya da Model'den View'a bir ilişki eklenir ve model değiştiğini bildirir. İkinci yol bağlaşımı 3'ten 4'e çıkarır ama View'u hafif tutar. Birden fazla View aynı veriye bakıyorsa Observer kalıbı kullanılabilir.

Dersin Toparlaması

Bu ilk derste dersin iki kısmı tanıtıldı: GoF tasarım kalıpları ile code smells ve refactoring. Derste başarılı olmak için bir NYP diline ve özellikle UML sınıf şemalarına hakimiyet gerekir. Kaliteli yazılımın iyi tasarımla mümkün olduğu, programcıya yönelik iç kalite özelliklerinin (yeniden kullanılabilirlik, ilgi alanlarının ayrılması, esneklik, taşınabilirlik, okunabilirlik, sınanabilirlik, bakım kolaylığı) bu dersin odağı olduğu anlatıldı. Proje başarı oranları, iyi tasarımın neden önemli olduğunu sayılarla gösterdi. Bağlaşım ve uyum, iyi tasarımı ölçmenin somut araçları olarak ele alındı: iki küçük örnek, bağlaşım değerinin ölçütün tanımına göre değiştiğini ve bazen daha yüksek uyum için daha yüksek bağlaşımın kabul edilebileceğini gösterdi.

İkinci yarıda tasarım kalıbının tanımı, dört bileşeni (isim, sorun, çözüm, tartışma/sonuçlar), GoF kalıplarının yaratımsal-yapısal-davranışsal ve sınıfsal-nesnesel sınıflandırmaları verildi. Örnek kalıp olarak MVC işlendi ve temel yapının ihtiyaca göre (Model → View, View → Controller ilişkileri, kalıtımla yeni Controller) nasıl değiştirilebileceği tartışıldı. Hoca bundan sonraki derslerde GoF kitabının sırasıyla, yaratımsal kalıplardan başlayarak ilerleneceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.